Blinking LED – A Look Under the Hood

1 Goal of this part of the project

Many people program microcontrollers using the Arduino IDE. There, all you have to do is type in your C/C++ code and click a button – the code is compiled and transferred directly to the microcontroller (flashed). Simple and convenient.

In this part of the project, we want to understand what exactly happens when you press that button. We set the Arduino IDE aside and write the machine code directly ourselves. That means we create the program file by hand – like real hackers or engineers who want to know what is really going on.

We will not be dealing with assembly language yet – that comes later. Instead, we write the machine code directly, byte by byte. This is something every technician or engineer should have done once in their life – if only to understand what a high-level programming language actually does for us. The real challenge here is not the machine code itself, but working with the terminal:

- How do you write a file in the editor `nano`?
- How do you organize your directories?
- How do you navigate in the terminal and display files?
- And finally, how do you transfer the machine code to the microcontroller with `avrdude`?

These are all important fundamentals – and after this part, you can say about yourselves: I have written a microcontroller program directly in machine code and flashed it myself.

But that is not all. The practical goal will be to make an LED blink. More precisely, it should alternately be on for 250 ns and off for 250 ns. You can no longer see this with the naked eye – so we will measure it with the oscilloscope. The period is

$$T = 250\,\text{ns} + 250\,\text{ns} = 500\,\text{ns},$$

which corresponds to a frequency of

$$f = \frac{1}{T} = \frac{1}{500\,\text{ns}} = 2\,\text{MHz}.$$

In our first experiment, we will see that we do generate a signal that switches the LED on and off, but it will not be symmetric. That is not a problem, though – it gives us the opportunity to learn a bit more about how the machine code is executed.

And because writing machine code is not difficult but tedious, we then switch to a higher-level language after all – not C/C++ yet, but first assembly language. With an assembler (`avr-as`), the code can be translated into machine code. Compared to C/C++, you can immediately see that you are actually doing exactly the same thing as in machine code, only more conveniently.[1]

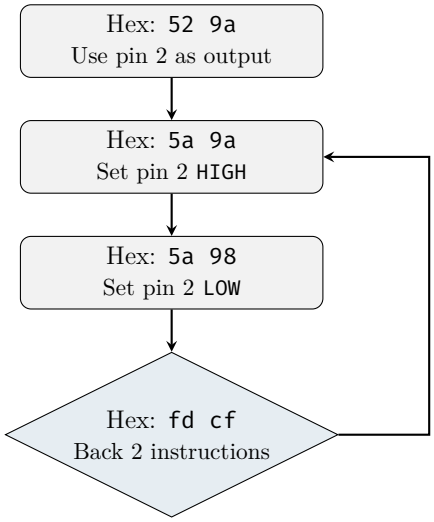
[1] By the same token, C/C++ is also just a convenient and portable way to express assembly code compactly.

2 A simple program to control an LED

To make an LED blink quickly, we need only four machine instructions:

- one to configure the desired pin of the microprocessor as an output,
- one to set this pin `HIGH` (i.e., to apply a voltage),
- one to set it `LOW` (i.e., to remove the voltage again),
- and a so-called *jump instruction*, which lets program execution continue at a different location.

Each of these instructions consists of two bytes and can therefore be represented by four hexadecimal digits. The instructions are executed one after another (sequentially) – as shown in the following flowchart:



Writing down the hexadecimal values of the instructions one after another gives us our first complete program in machine code:

```
52 9a 5a 9a 5a 98 fd cf
```

Each hex digit stands for exactly four bits. The complete bit patterns can be looked up in Table 1. In binary notation, our program then looks like this:

```
01010010 10011010 01011010 10011010 01011010 10011000 11111101
11001111
```

The program is now completely designed. Only one last step is missing: it has to be written into the program memory of the microprocessor – that is, into the so-called *flash* memory.
The next section explains how to write the machine code in hexadecimal notation into a file and transfer it to the flash memory of the microprocessor with the program `avrdude`.

What is done today with a single command was considerably more laborious with historical computers and microprocessors of the 1950s to 1970s: programs were often

Table 1: *All 4-bit numbers in decimal, hexadecimal, and binary notation*

Decimal	Hexadecimal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
10	A	1010
11	B	1011
12	C	1100
13	D	1101
14	E	1110
15	F	1111

loaded bit by bit, by flipping a toggle switch for each individual bit (binary digit). [2] Alternatively, programs were punched onto punch cards and read in mechanically – which in principle also meant nothing more than entering many bits in a row, just somewhat more conveniently. Today, these old methods make a good mental model for better understanding what the program `avrdude` actually does for us behind the scenes.

[2]Entering programs via toggle switches was perfectly normal in the 1970s – from the Altair 8800 in the hobbyist's basement to the legendary supercomputers of Seymour Cray, the pioneer of high-performance computing. On the Altair, you had to key in the bootloader bit by bit on the front panel, and LEDs showed the current memory contents. Cray, too, entered the first code by hand on his early machines before magnetic tape drives loaded larger programs.

3 Transferring the program to the microprocessor ("flashing" the microprocessor)

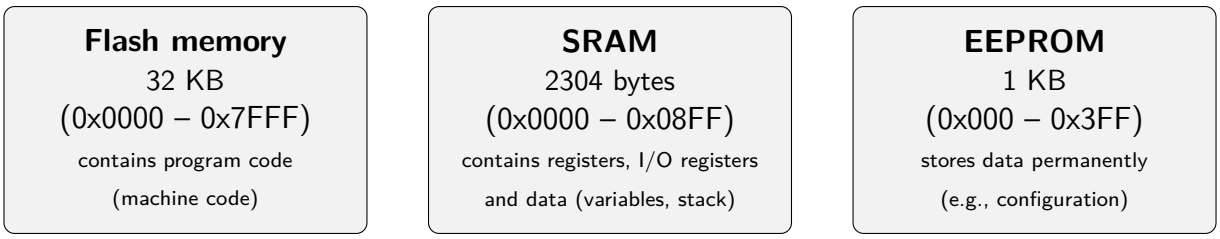
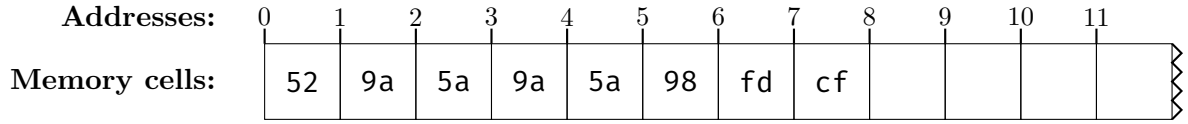


Figure 1: *Memory organization of the ATmega328P: the flash contains the program, the SRAM contains registers, I/O areas, and data. The EEPROM stores non-volatile user data.*

Our microprocessor has three different kinds of memory (Figure 1):

- A memory for the program – the so-called *flash*. The data stored there are retained even when you pull the plug.
- A *volatile memory* (RAM) that is available during program execution. The data in it are lost as soon as the power is gone.
- A so-called *EEPROM* (Electrically Erasable Programmable Read-Only Memory), in which small amounts of data can be stored permanently – e.g., configurations or states that should be retained even after switching off.

For now, only the flash memory matters to us – this is where our program has to be transferred. All kinds of memory have one thing in common: they consist of arrays of memory cells. Each cell can store exactly one byte, i.e., 8 bits. These 8 bits can be represented compactly by two hexadecimal digits. The memory cells are numbered consecutively – this number is called the *address*. The first address always has the value 0. Our goal is for the program memory to look like this after flashing:



Starting at address 0, the individual bytes should thus lie in memory one after another. Once this has been achieved, program execution can start – it automatically begins with the instruction stored at address 0.

After an instruction has been executed, typically the instruction located two addresses further on in memory is executed next – because on our microprocessor every instruction consists of exactly two bytes. Jump instructions are an exception: they can explicitly specify that execution continues at a different address – either forward (skipping some instructions) or backward (jumping back).

3.1 Intel HEX format

A tool that transfers the machine code to the flash memory must know at least two things about the data to be written:

1. How many bytes have to be transferred in total.
2. At which address in the flash memory the first byte should be written. The following bytes are then automatically stored at the subsequent addresses.

This information must be provided in a suitable form – typically by means of a special file in which the machine code and the memory addresses are encoded. A common format for this is the so-called *Intel HEX format*, which is understood by many programs – including `avrdude`.

Our machine code can be packed as follows into a file `led.hex` that uses this format:

```
:08000000529A5A9A5A98FDCF5A
:00000001FF
```

Listing 1: *File `led_pin2.hex`*

In this case, the file `led.hex` contains two so-called *records*, each shown as one line. Each record begins with a colon (`:`) and consists of a series of hexadecimal digits that are interpreted in groups:

Data line is a so-called *data record*, which contains the actual machine code bytes and specifies at which address in memory they should be stored.

`:` `08` `0000` `00` `529A5A9A5A98FDCF` `5A`

Number of data bytes:	08
Start address in memory:	0000
Record type:	00 (data)
Data bytes:	52 9A 5A 9A 5A 98 FD CF
Checksum:	5A

Here once more is the meaning of the individual parts in detail:

- `08` specifies the number of data bytes in this record (here: 8 bytes).
- `0000` is the start address in the flash memory at which the following data should be stored (in this case, address 0).
- `00` is the record type. `00` means “normal data block”.
- `52 9A 5A 9A 5A 98 FD CF` are the 8 bytes of machine code that are to be written.
- `5A` is a checksum for error detection.

End-of-file line is a so-called *end-of-file record*. It signals to the flash tool that no further data follow. The structure is similar to that of the previous record:

```
: 00 0000 01 FF
```

Number of data bytes:	00
Start address in memory:	0000 (not relevant)
Record type:	01 (end)
Data bytes:	(none)
Checksum:	FF

The meaning of the individual parts in summary:

- `00` — There are no data bytes in this record.
- `0000` — Start address (irrelevant, since no data follow).
- `01` — Marks this record as end-of-file.
- `FF` — The checksum, which here, too, serves as a check.

This format is very compact and easy for machines to process. Programs such as `avrdude` read this file and automatically write the data it contains to the specified addresses in the flash memory of the microprocessor.

3.2 Intel HEX format: computing the checksum

The checksum is used for error detection. For each record, the microprocessor (or the flash tool) first receives the information on how many data bytes it contains, followed by the actual bytes – and finally the checksum. After reception, the checksum can be computed again and compared with the transmitted one. If they do not match, it is clear that an error occurred during transmission – either in the data themselves or in the checksum.

If, on the other hand, the checksum matches the computed one, this does *not* mean that no error is guaranteed to have occurred – only that none was detected. The checksum therefore merely reduces the probability that a transmission error goes unnoticed.

As with all checksum methods, a compromise has to be made here:

- If the probability of undetected errors is to be as low as possible, the checksum usually becomes more complex (e.g., through longer or cryptographic methods).
- If, on the other hand, the checksum is to be particularly simple and fast to compute (as in the Intel HEX format), the probability that certain errors go undetected increases – for example, when two faulty bytes happen to "cancel each other out".

The checksum in the Intel HEX format is a good compromise: very easy to compute, but sufficiently reliable for typical flashing over stable connections[3].

Computing the checksum in the Intel HEX format

The Intel HEX format uses a so-called *two's complement checksum*. Put simply: first, all bytes (except the checksum itself) are added up, using an 8-bit adder – that is, only the lower 8 bits of the sum are kept. The checksum is then exactly the number that would have to be added to this sum so that the result (computed with 8 bits) is `0` again.
For those who like to show off with formalism: for $a, b \in B = \{0, \ldots, 2^8 - 1\} = \{0, \ldots, 255\}$, the 8-bit addition is defined by $a \oplus b := (a + b) \bmod 2^8$. For the n bytes $b_1, \ldots, b_n$, one then first computes

$$s = b_1 \oplus b_2 \oplus \cdots \oplus b_n.$$

With respect to $\oplus$, the set B is a finite abelian group. Hence, for every $s \in B$ there exists an inverse element $-s$, and this $-s$ is the checksum p we are looking for. Thus:

$$p = -s \quad \Leftrightarrow \quad p \oplus s = 0,$$

so p is exactly the number that has to be added to the sum s to obtain 0 again (modulo 2^8).

For an example calculation by hand, you can compute directly in hexadecimal notation. This turns an 8-digit binary calculation into a much more compact 2-digit hexadecimal calculation.

To simplify phrases such as "the hexadecimal representation `19`", we will use the following convention from now on: a prefix `0x` indicates that the digits following it are to be read in the hexadecimal system.

So `0x19` means the number 25 in the decimal system, because:

$$0\text{x}19 = 1 \cdot 16 + 9 = 25.$$

Suppose we want to compute the checksum for the three bytes

$$0\text{xD8} \quad 0\text{xB5} \quad 0\text{x19};$$

then we proceed as follows:

1. First, we compute the sum of the three bytes using 8-bit addition:

$$s = (0\text{xD8} \oplus 0\text{xB5}) \oplus 0\text{x19}$$

The addition is carried out step by step. First:

```
  0xD8
+ 0xB5
  ----
  0x18D
```

[3]For high-integrity transmissions (e.g., in networks or file systems), CRCs (cyclic redundancy checks) or cryptographic hashes are often used instead of simple checksums.

Since we only compute with an 8-bit adder, we truncate the result to the lower 8 bits:

$$0\text{xD8} \oplus 0\text{xB5} = (0\text{xD8} + 0\text{xB5}) \bmod 2^8 = 0\text{x18D} \bmod 2^8 = 0\text{x8D}$$

Then:

```
  0x8D
+ 0x19
  ----
   1
  0xA6
```

Altogether, this gives $s = 0\text{xA6}$.

2. The checksum p is obtained by subtracting s from $2^8 =$ `0x100` and, if necessary, truncating the result to two digits (i.e., 8 bits). The truncation is only relevant in the special case $s = 0$, because then the calculation yields $p = 0x100$, but the correct value is of course $0x00$.

```
  0x100
- 0xA6
  -----
   1 1
  0x5A
```

This calculation of $p = (2^8 - s) \bmod 2^8$ corresponds exactly to the well-known method for forming the *two's complement*: you invert the bit pattern of s and then add 1.

For our example, this can be not only easily verified but even derived: we try to determine the number that has to be added to `0xA6` to obtain `0x100`. Along the way, we also introduce the prefix `b` to indicate a binary representation. We have:

$$s = \texttt{0xA6} = \texttt{b10100110} \quad \Rightarrow \quad \overline{s} = \texttt{b01011001} = \texttt{0x59}$$

Here, $s + \overline{s} = 2^8 - 1 = 0\text{xFF}$ always holds, as can easily be verified in this case:

```
 b10100110
+b01011001
----------
 b11111111
```

From this, it follows that:

$$s + \overline{s} = 2^8 - 1 \quad \Rightarrow \quad s + (\overline{s} + 1) = 2^8 \quad \Rightarrow \quad s \oplus (\overline{s} \oplus 1) = 0$$

In other words: $\overline{s} \oplus 1$ is exactly the number that has to be added to s in 8-bit arithmetic to obtain 0 – and therefore

$$p = \overline{s} \oplus 1 = \texttt{0x59} \oplus 1 = \texttt{0x5A}.$$

3.3 Flashing the microcontroller with `avrdude`

On Linux systems, the command `ls /dev/ttyUSB*` lists the devices that are recognized as serial interfaces – for example, a microcontroller connected via USB. The device name found here is needed later when using `avrdude`, in order to specify the target port correctly. Typically, you get output like this:

```
/dev/ttyUSB0
```

> **Note**
>
> If several devices are connected, it can be helpful to run the command `ls /dev/ttyUSB*` once before and once after plugging in the Arduino, in order to identify the correct device name.

The program `avrdude` can then be used to transfer the machine program stored in `led.hex` to the flash memory of the microcontroller. The complete command looks roughly like this:

```
avrdude -c arduino -P /dev/ttyUSB0 -p atmega328p -U flash:w:led.hex:i
```

The options stand for:

- `-c arduino`
  Specifies which programmer is used. In this case, the Arduino bootloader is addressed, which can be programmed via the serial interface.
- `-P /dev/ttyUSB0`
  Specifies the device through which communication takes place. This is the name of the serial interface determined previously with `ls /dev/ttyUSB*`.
- `-p atmega328p`
  Specifies the target processor, here the ATmega328P (as found on an Arduino Uno).
- `-U flash:w:led.hex:i`
  Means: the flash memory (`flash`) is to be written (`w` = write) with the data from the file `led.hex`. The `i` at the end stands for the file format – here: Intel HEX.

4 Measuring what the program does

As soon as the program has been written to the flash memory, the microcontroller automatically starts executing it.

If you connect an LED to pin 2, it lights up – but with the naked eye you cannot tell that it is blinking extremely fast. Not even a slight flicker or a perceptible change in brightness is visible.

To check what actually happens, we use an oscilloscope in the lab. Figure 2 shows the measurement of the signal at pin 2.

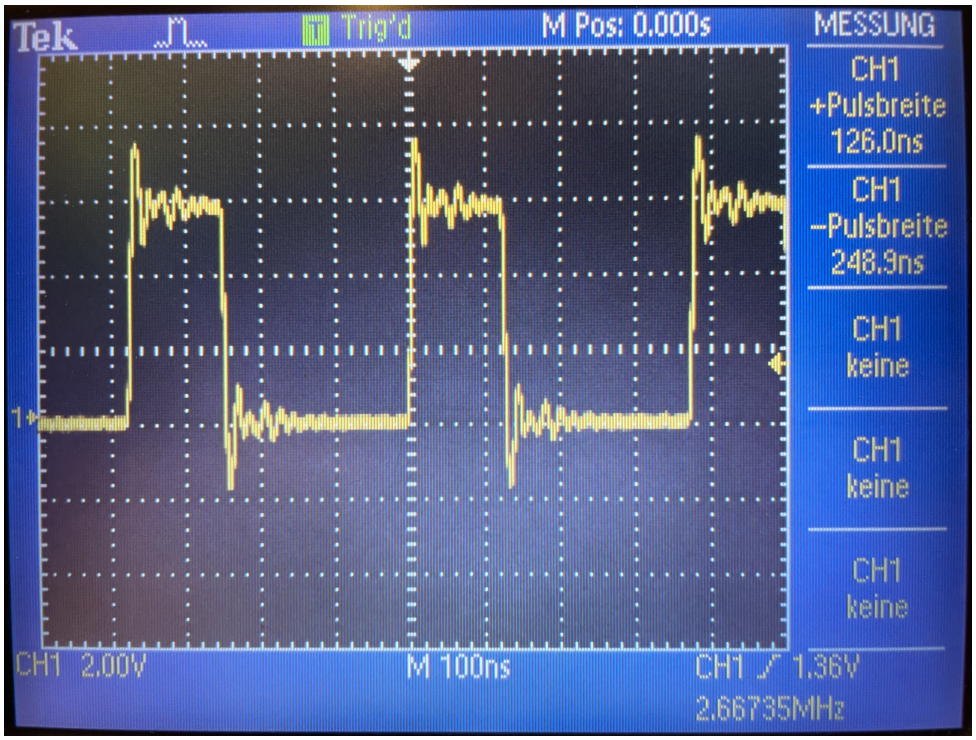


Figure 2: *Oscilloscope measurement at pin 2 while* `led.hex` *is running. The HIGH phase lasts about 125 ns, the LOW phase about 250 ns – the signal is therefore not symmetric.*

First, the good news: our program works as expected – the pin is alternately set `HIGH` and `LOW`.

However, the measurement also shows that the signal is not symmetric. The HIGH phase lasts only about 125 ns, while the LOW phase is about twice as long.

Why this is so is analyzed in the next section – based on the internal clock cycles that each instruction requires.

5 Analyzing the program execution

The microcontroller used in the lab (ATmega328P) runs at a clock frequency of 16 MHz. A single CPU clock cycle therefore lasts:

$$\frac{1}{16\,\mathrm{MHz}} = \frac{1}{16 \cdot 10^6}\,\mathrm{s} = \frac{1000}{16} \cdot 10^{-9}\,\mathrm{s} = 62.5\,\mathrm{ns}.$$

The ATmega328P datasheet specifies how many clock cycles the individual machine instructions require. For our example, there is a pleasing regularity: each of the instructions used takes exactly two clock cycles to execute – including the jump instruction. We therefore add the respective number of cycles to our flowchart:

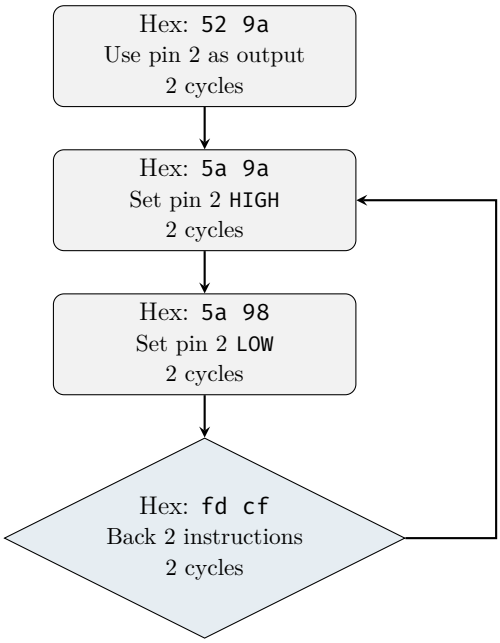


Figure 3 shows how the state of pin 2 changes over the course of program execution. If you keep in mind that two CPU clock cycles correspond to 125 ns and four cycles to 250 ns, this explains not only the waveform measured in Figure 2. It also shows that, with exact knowledge of the number of cycles per instruction, you can predict very precisely how the signal will behave – without measuring at all.

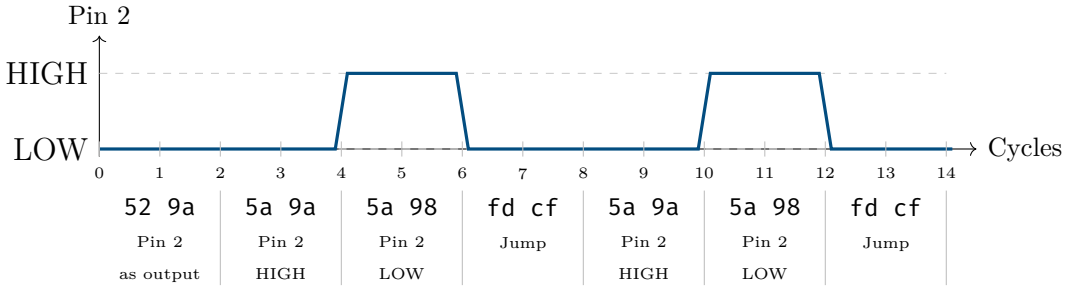


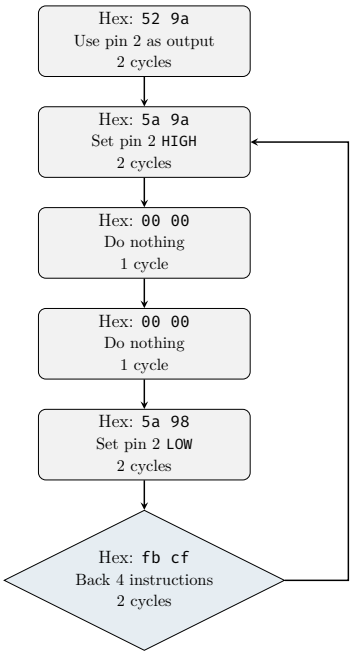
Figure 3: *Signal at pin 2 as a function of the CPU clock cycles. The HIGH and LOW states result from the machine instructions used, each of which takes two clock cycles.*

In the next section, we will modify the program so that a symmetric signal is produced.

6 Modifying the program for a symmetric signal

To produce a symmetric signal, all we have to do is make sure that pin 2 stays `HIGH` for two additional clock cycles. Fortunately, the ATmega328P – like almost every processor – provides a so-called `NOP` instruction ("No Operation"), which does exactly that: nothing. According to the datasheet, this instruction takes exactly one CPU clock cycle.

We can therefore easily modify our existing program by inserting two `NOP` instructions directly after setting pin 2 `HIGH`. This keeps the pin `HIGH` for a total of four clock cycles (instead of two as before), and we get a symmetric signal with `HIGH` and `LOW` phases of equal duration. The new flowchart looks like this:



Writing down the hexadecimal values of the instructions one after another, the modified program reads:

```
52 9a 5a 9a 00 00 00 00 5a 98 fb cf
```

Finally, the question arises of how the modified program can be converted into the Intel HEX format. The challenge is to represent the new instructions correctly in a hex file so that `avrdude` can write them to the flash memory without errors.

Exercise

Create a valid Intel HEX file for the modified program. Pay attention to correct addressing and to computing the checksum correctly. Then you can transfer the new program to the microcontroller with `avrdude` and use the oscilloscope to check whether a symmetric signal is now produced.

How you can quickly compute the checksum in this case in the lab under battlefield conditions is explained in Lab Handout 1.

7 Assembly language instead of machine code

We wrote the previous program for controlling the LED directly in machine code – that is, we specified byte by byte which instructions are to be executed. That was an instructive experience, but also a tedious one. Fortunately, there are tools that do this work for us. A so-called *assembler* automatically translates programs written in a human-readable symbolic language (so-called *assembly code*) into machine code. An assembly program that produces exactly the same behavior as our modified LED program could look like this:

```
      sbi   0x0a, 2  ; Configure pin 2 as output

loop: sbi   0x0b, 2  ; Set pin 2 HIGH
      nop            ; Do nothing
      nop            ; Do nothing
      cbi   0x0b, 2  ; Set pin 2 LOW
      rjmp  loop     ; Back to label 'loop' (i.e., 4 instructions back)
```

Listing 2: *File* `led_pin2.s`

Here:

- `sbi` stands for "Set Bit" – a specific bit in an I/O register is set to `1`.
- `cbi` stands for "Clear Bit" – a specific bit is set to `0`.
- `nop` is a so-called no-operation – the instruction does nothing but takes up one CPU clock cycle.
- `rjmp` is a relative jump instruction – it jumps back by a certain number of instructions (here: to the label `loop`).

The advantage: the code is much more readable than a sequence of bytes such as `52 9a ...`. You can see immediately what is meant – and the risk of errors is much lower.

7.1 From assembly code to an Intel HEX file

An assembly program like the one shown above can be translated into machine code with the tool `avr-as`. In the good old days, that was often all there was to it: the generated machine code could be executed directly or saved as an Intel HEX file – done. The world was simple, and so were the tools.

Today, systems are more complex – and so are the toolchains. To deal with this complexity, the tasks are divided into smaller steps:

- `avr-as` first generates a so-called *object file* (`.o`) with machine code, but still without complete address information.
- This file is further processed by the *linker* `avr-ld`. The linker replaces symbolic jump targets – such as the label `loop` in our assembly program – with concrete addresses. This produces an executable file in the so-called *ELF format* (Executable and Linkable Format), a modern standard on Linux.

- Since the ATmega328P of course does not understand the ELF format (after all, it comes from the good old days and, as is only proper, expects raw machine code), the machine code has to be extracted from the ELF file with `avr-objcopy` and converted into the *Intel HEX format* – a format suitable for flashing the microcontroller.

The complete toolchain thus looks like this:

```
avr-as -o led_pin2.o led_pin2.s
avr-ld -mavr5 -o led_pin2.elf led_pin2.o
avr-objcopy -O ihex -R .eeprom led_pin2.elf led_pin2.hex
```

Before flashing the microcontroller, you can look at the contents of the file `led_pin2.hex` – it should match exactly the Intel HEX file created by hand in Section 6. The microcontroller can then be flashed with `avrdude`, as already shown in Section 3.3.

7.2 A Makefile for Linux and macOS

To make working with several assembly files easier, we use the `Makefile` shown in Listing 4. It ensures that a matching Intel HEX file is automatically generated for every `.s` file. For this, the Makefile should be located in the same directory as the assembly files. To translate, all you need is the command:

```
make
```

In addition, with

```
make upload HEX=led_pin2.hex
```

you can transfer a specific hex file – such as `led_pin2.hex` here – to the microcontroller. In principle, the "Verify" button in the Arduino IDE hides a call to `make`, and the "Upload" button a call to `make upload HEX=…`.

For the translation, the toolchain described above, consisting of `avr-as`, `avr-ld`, `avr-objcopy`, and `avrdude`, is used – but in a sensible combination. More precisely:

- `avr-as` generates `led_pin2.o` only if `led_pin2.s` has changed – that is, if the source file is newer than the object file or the latter does not exist yet.
- `avr-ld` generates a new `led_pin2.elf` only if `led_pin2.o` has changed.
- `avr-objcopy` generates a new `led_pin2.hex` only if `led_pin2.elf` has changed.

This saves unnecessary work – and that is precisely the strength of `make`. If, on the other hand, a file is uploaded directly with

```
make upload HEX=led_pin2.hex
```

Listing 4 File `Makefile` for translating assembly programs on Linux and macOS. This version processes `.s` files only.

```
# This version only processes assembly programs (*.s)
# Later, an extended Makefile will also support C.

SOURCES := $(wildcard *.s)
OBJECTS := $(SOURCES:.s=.o)
ELFS  := $(SOURCES:.s=.elf)
HEXS  := $(SOURCES:.s=.hex)

all: $(HEXS)

%.o: %.s
      avr-as -o $@ $<

%.elf: %.o
      avr-ld -mavr5 -o $@ $<

%.hex: %.elf
      avr-objcopy -O ihex -R .eeprom $< $@

clean:
      $(RM) $(OBJECTS) $(ELFS) $(HEXS)

ifeq ($(MAKECMDGOALS),upload)
ifeq ($(shell uname), Linux)
PORT := $(shell ls /dev/ttyUSB* 2>/dev/null | head -n 1)
else
PORT := $(shell ls /dev/cu.usb* 2>/dev/null | head -n 1)
endif # uname
endif # MAKECMDGOALS == upload

upload: $(HEX)
      @if [ -z "$(HEX)" ]; then \
         echo "Usage: make upload HEX=<filename.hex>"; \
         false; \
      fi
      @if [ -z "$(PORT)" ]; then \
         echo "no USB device found"; \
         false; \
      fi
      avrdude -c arduino -P $(PORT) -p atmega328p -U flash:w:$(HEX):i
```

then `make` automatically checks whether the specified file is up to date – that is, whether steps such as translating from `.s` to `.o`, linking to the `.elf` file, or generating the `.hex` file might be required first. These dependencies, too, are reliably detected and executed in the correct order – without any further action on your part.

Even if you would not suspect it at first glance: what you see in the `Makefile` is actually a programming language. Its syntax and user-friendliness are certainly debatable – but it is functional, no doubt: it automatically detects dependencies between files, decides which intermediate steps are necessary, and executes them in the correct order.

However, some things hurt to look at – for example, the block in lines 23–29. The fact that the `ifeq` constructs there are not indented is not an oversight but, unfortunately, necessary obedience: `make` accepts directives such as `ifeq`, `endif`, `define`, etc. only at the beginning of a line – just like simple assignments such as `PORT := ...`. Visual indentation for better readability is simply not allowed. Especially if you – like the author – are allergic to inconsistent indentation in C and consider it a blessing that Python enforces indentation, the syntax of `make` sometimes feels like a little journey back in time to the 1980s.

The Makefile in Listing 4 will be provided to you in the lab. How to extend it so that C programs can be processed as well will be part of a lab task later on. It is not necessary to know every detail about Makefiles for this. To begin with, it is enough if you understand how the dependencies described above are implemented in the Makefile.

Line 11, for example, contains the rule for how to generate a `.o` file from a `.s` file – provided that an update is necessary (i.e., the `.o` file is missing or older than the corresponding `.s` file). Line 12 contains the corresponding command, `avr-as -o $@ $<`, where `$@` is automatically replaced by the name of the target file (i.e., the `.o` file) and `$<` by the name of the source file (i.e., the `.s` file).

Altogether, the Makefile thus describes a directed graph of dependencies, as visualized in Figure 5: each step builds on the previous one, and `make` ensures that all necessary commands are executed in the correct order – but only when they are really required.

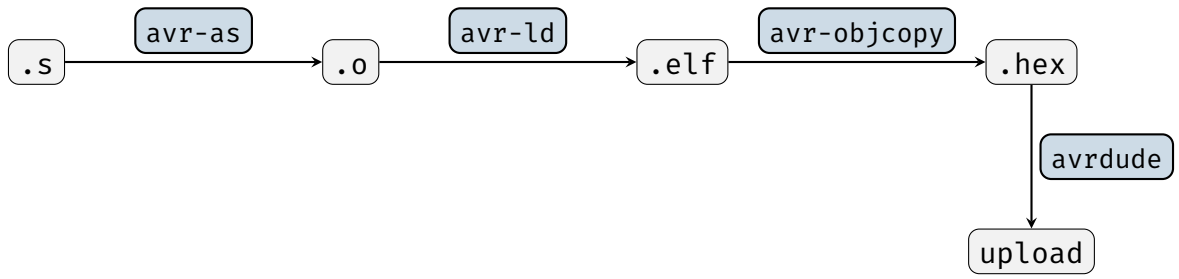


Figure 5: *Dependency graph for generating a `.hex` file and the subsequent upload. The tools `avr-as`, `avr-ld`, and `avr-objcopy` generate the `.hex` file, which is finally transferred to the microcontroller by `avrdude`.*

7.3 Blinking more elegantly with exclusive OR

The following program – like the previous one – makes an LED blink so that it is alternately on for 250 ns and off for 250 ns. So once again, a clean, symmetric signal is produced.

The difference lies in the code: this time, registers are used – and, above all, a new instruction, namely an exclusive OR. This logical operation is often abbreviated as **XOR** (for *exclusive OR*). On the ATmega328P, however, the corresponding machine instruction is called **EOR** – as in line 5 of the following program.

Let us first take a look at the code before we work out in detail how it works and what role registers and other special features play:

```
1       ldi   r19,  4        ; r19 = mask for pin 2 (note: 4 = 2^2)
2       out   0x0a, r19      ; pin 2 as output
3       ldi   r18,  0        ; r18 = 0
4 loop:
5       eor   r18,  r19      ; r18 = r18 XOR r19 ("toggle")
6       out   0x0b, r18      ; set pin 2 HIGH or LOW
7       rjmp  loop
```

Listing 3: *File* `led_pin2_xor.s`

Before we analyze the new code step by step, let us first clarify some important terms and concepts:

- What are registers – and how do they differ from normal memory cells?
- What exactly are I/O registers – and how can they be used to control the pins of the microcontroller?
- How can a pin be configured as an input or output?
- And how can it be set specifically `HIGH` or `LOW`?

The following sections answer all of these questions. They not only help you understand the new code better but, in retrospect, also explain many details of our very first assembly program (Section 7).

By the way, in the lab you are welcome to simply try out first whether this new program really produces a 250 ns–250 ns blink signal again – just like the previous one. If the oscilloscope shows a familiar pattern, that is already a good sign. And now let us look in detail at how exactly this program works.

Registers: computing only with registers Registers are special memory cells that the processor can use directly for arithmetic operations. The ATmega328P has 32 of them – each 8 bits wide – named `r0`, `r1`, `r2`, …, `r31`. Technically, on the ATmega328P the registers are located in the same data space as the SRAM (Figure 1), which is also used for local and global variables – *the exact structure of this memory area is shown in* Figure 6.

The special thing is: arithmetic operations such as addition, comparison, or logical operations can be performed exclusively with registers. If you want to work with variables located in normal SRAM, you first have to load them into a register – and, if necessary, write them back after the computation.

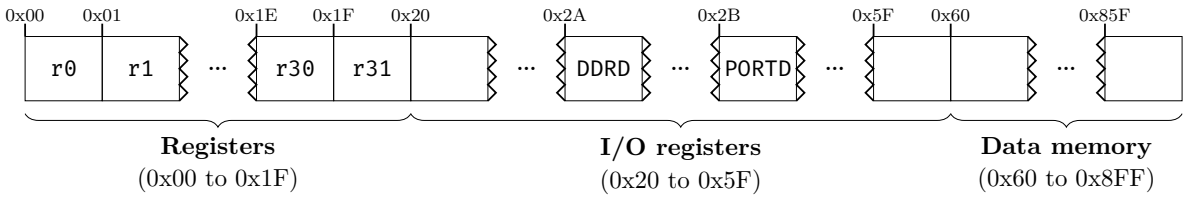


Figure 6: *Schematic representation (not to scale) of the SRAM area of the ATmega328P. Shown are the memory cells for the registers as well as the memory addresses with which the direction of the pins can be set (e.g., DDRD) or the outputs can be set directly (e.g., PORTD).*

To be able to compute with registers at all, they first have to be assigned a value. A simple way to do this is the instruction `ldi`, which stands for "load immediate", i.e., "load directly". It lets you write a constant directly into a register:

```
ldi   r19, 4   ; r19 gets the value 4
ldi   r18, 0   ; r18 gets the value 0
```

In this example, register `r19` is preloaded with the value `4` – i.e., with a bit mask in which only bit 2 is set. Register `r18` gets the value `0`.

Now a typical arithmetic operation comes into play: the so-called *exclusive OR* operation (XOR), which is implemented in the ATmega328P assembly language by the instruction `eor`. It works bitwise and yields a `1` only if the two operands have different bits at the respective position. The instruction

```
eor   r18, r19
```

causes the contents of `r18` to be replaced by `r18 XOR r19`. Since `r18` previously had the value `0`, and `r19` has the value `4`, the result is `r18 = 0 XOR 4 = 4`. If the instruction

```
eor   r18, r19
```

is then executed again, `r18 = 4 XOR 4 = 0`. The original value has thus been cleared again. Repeated application of this instruction therefore toggles the contents of `r18` between `0` and `4` – exactly this behavior is what we use to set the pin state alternately `HIGH` and `LOW`:

```
    00000100       (r19 = 4)
XOR 00000000       (r18 = 0)
-------------------------------
    00000100  (new: r18 = 4)
```

```
    00000100       (r19 = 4)
XOR 00000100       (r18 = 4)
-------------------------------
    00000000  (new: r18 = 0)
```

I/O registers: the interface to the pins I/O registers (input/output registers) are special memory cells in the SRAM area through which the pins of the microcontroller can be controlled. For pins 0–7 of our microprocessor, the addresses `0x2A` and `0x2B` are particularly relevant:

- `0x2A` corresponds to the so-called `DDRD` register. If the bit with index 2 is set there, pin 2 is configured as an output.

- `0x2B` corresponds to the `PORTD` register. If the bit with index 2 is set there, pin 2 is at a HIGH level – otherwise it is LOW.

Interestingly, we already used these registers in our first assembly program (see Section 7) – but with different addresses. There we had, for example:

```
sbi   0x0a, 2 ; Configure pin 2 as output
sbi   0x0b, 2 ; Set pin 2 HIGH
cbi   0x0b, 2 ; Set pin 2 LOW
```

Here, the address `0x0a` was used instead of `0x2A`, and `0x0b` instead of `0x2B`.
The reason: the instructions `sbi` (set bit) and `cbi` (clear bit) are specifically intended for accessing I/O registers directly. They operate only on addresses in the range `0x00` to `0x1F`, which corresponds to the first 32 I/O registers. Internally, an offset of `0x20` is automatically added when these instructions are executed. This saves valuable bits in the machine instruction, since only a small address range has to be encoded.

Overwriting an I/O register completely In the previous example, individual bits in the I/O registers were specifically set or cleared – e.g., using the instructions `sbi` (set bit) or `cbi` (clear bit). An alternative is to overwrite the entire register at once. This is not only more compact and clearer but also more efficient: while `sbi` and `cbi` each take two CPU clock cycles, a complete write access with the instruction `out` takes only a single cycle.

As a reminder:

- The register `DDRD` (address `0x2A`) determines which pins are used as outputs. To enable only pin 2 as an output, you write the number `4` ($= 2^2$) into the `DDRD` register – that is, exactly the bit with index 2.
- The register `PORTD` (address `0x2B`) determines whether an output pin is `HIGH` or `LOW`. To set pin 2 `HIGH`, you likewise write the value `4` into it. To set it `LOW`, the value `0` is sufficient.

For this, you use the instruction `out`, which requires two operands:

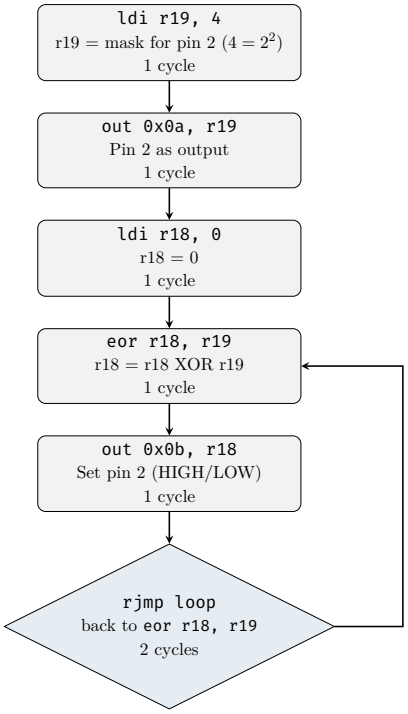
- the target address – e.g., `0x2A` for `DDRD` or `0x2B` for `PORTD`
- a register that contains the value to be written

This is exactly what the following example does:

```
ldi   r19, 4     ; set bit 2
out   0x0A, r19  ; configure pin 2 as output (DDRD)
out   0x0B, r19  ; set pin 2 HIGH (PORTD)
```

Important: with `out`, too – as with `sbi` and `cbi` – an offset of `0x20` is added internally to the specified address. In the example above, the writes therefore actually go to address `0x2A` and address `0x2B`, respectively, which are exactly the physical memory addresses of `DDRD` and `PORTD` (see Figure 6).

We have now understood every instruction in Listing 3. If we now also analyze the program flow, it becomes clear why the signal at pin 2 is symmetric again – that is, as with the first program, 250 ns `HIGH` and 250 ns `LOW` each. The following flowchart shows the number of cycles and explains how the pin state changes over time:


ldi r19, 4
r19 = mask for pin 2 (4 = 2^2)
1 cycle
out 0x0a, r19
Pin 2 as output
1 cycle
ldi r18, 0
r18 = 0
1 cycle
eor r18, r19
r18 = r18 XOR r19
1 cycle
out 0x0b, r18
Set pin 2 (HIGH/LOW)
1 cycle
rjmp loop
back to eor r18, r19
2 cycles

8 A simple delay with one register

Our next goal is to make pin 2 switch between HIGH and LOW somewhat more slowly. Ultimately, it should be possible to connect an LED there that then blinks at a given frequency that is perceptible to the eye.

A little spoiler: with the current approach, the LED would blink at about 20,000 hertz – that would still not be perceptible to the eye (even if it were technically possible to switch an LED on and off at such a high frequency).

But the idea we develop and implement here can easily be improved to reach our goal in the next section. Compared to Listing 3, the code shown in Listing 4 contains three additional instructions, which have been added in lines 7 to 10:

- The instruction in line 7 initializes register `r24` with the value 2. This register serves as a counter for a countdown and can of course also be loaded with other values. However, since registers on our microcontroller are only 8 bits wide, only values from `0` to `255` (or, in hexadecimal notation, from `0x00` to `0xFF`) are possible.
- The instruction in line 9 decrements register `r24` by 1. Technically, the value `1` is subtracted from `r24` and the result is stored back in `r24`. What matters here is that this subtraction computes not only the arithmetic result but also so-called *status flags*. One of them is the so-called *zero flag*, which is set exactly when the result of the subtraction equals 0.
- The instruction in line 10 is a conditional jump. `brne` stands for "Branch if Not Equal" and means: the jump to the label `delay` is taken only if the zero flag is *not* set – that is, exactly when `r24` is not yet 0 after the last subtraction. If, on the other hand, the zero flag is set (i.e., `r24` is now 0), execution continues with the next instruction in line 12.

Figure 7 shows a flowchart for the program in Listing 4. It makes clear that the newly inserted lines implement an inner loop in which register `r24` is counted down. This loop is embedded in an outer loop that sets pin 2 alternately `HIGH` and `LOW`.

The larger the initial value of `r24`, the more cycles are needed to run through the inner loop before the state of pin 2 is changed again. The time that pin 2 remains in a particular state (i.e., `HIGH` or `LOW`) can thus be controlled deliberately.

8.1 Clock cycles per loop iteration

We now want to compute how many clock cycles elapse between two consecutive executions of the `eor` instruction in line 5 – that is, how long it takes until the state of pin 2 changes again.

The decisive factor is how often the inner loop is executed. If register `r24` is initialized with a value n, the loop is executed n times in total. The jump back to the label `delay` is taken exactly $n-1$ times. The n-th time, the result of the subtraction is zero, the zero flag is set, and no jump is taken anymore.

```
1       ldi   r19,  4         ; 1 cycle
2       out   0x0a, r19       ; 1 cycle
3       ldi   r18,  0         ; 1 cycle
4 loop:
5       eor   r18,  r19       ; 1 cycle
6       out   0x0b, r18       ; 1 cycle
7       ldi   r24,  0x2       ; 1 cycle (set countdown counter)
8 delay:
9       subi  r24,  1         ; 1 cycle
10      brne  delay           ; 2 cycles if jump taken, 1 cycle otherwise
11
12      rjmp  loop            ; 2 cycles
```

Listing 4 `led_pin2_delay1.s` – pin 2 is toggled with a delay by means of a countdown loop in `r24`

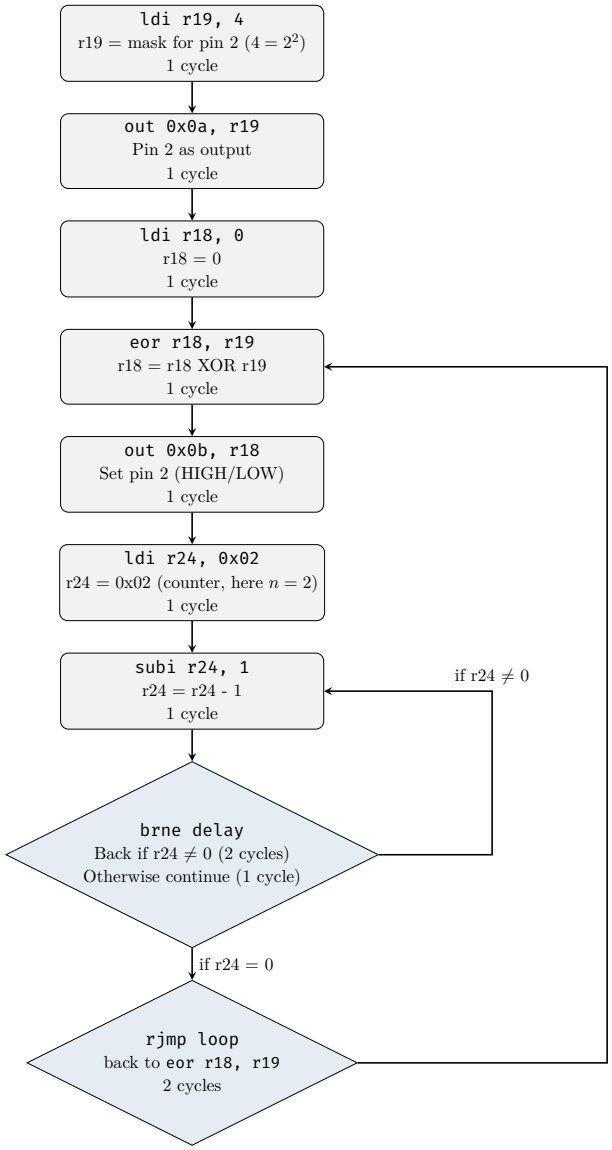


Figure 7: *Flowchart for the program from Listing 4 with a delay loop using* `r24`*. The outer loop changes the pin state, the inner loop produces a variable pause.*

According to the datasheet, the `subi` instruction (line 9) always takes 1 cycle. The `brne` instruction (line 10) takes:

- 2 cycles if the jump is taken ($n-1$ times),
- 1 cycle if no jump is taken (once).

The inner loop (lines 9–10) thus takes a total of:

$$(n-1)\cdot(1+2)+1+1 = 3(n-1)+2 \quad \text{cycles}$$

In addition, the following instructions are executed before and after this loop:

- Line 7: `ldi r24, n` $\rightarrow$ 1 cycle
- Line 6: `out 0x0b, r18` $\rightarrow$ 1 cycle
- Line 5: `eor r18, r19` $\rightarrow$ 1 cycle
- Line 12: `rjmp loop` $\rightarrow$ 2 cycles

This gives a total of:

$$3(n-1)+2+3+2 = 3n+4 \quad \text{cycles per loop iteration.}$$

So the larger the initial value n, the longer it takes until the state at pin 2 changes again.

8.2 Measuring the delay

The following table shows how many clock cycles are needed for one iteration of the outer loop, depending on the initial value n in register `r24`. According to the previous derivation, the number of cycles is:

$$T(n) = 3(n-1)+7 = 3n+4 \tag{1}$$

On a microcontroller running at 16 MHz, one clock cycle lasts exactly 62.5 ns. For various values of n, Table 2 shows how many cycles and how much time a complete iteration of the outer loop takes. Of course, we will measure this in the lab. It is helpful to first check, for small values of n, how long pin 2 stays HIGH and LOW, respectively, in order to validate the formula from (1).

Initial value n	Cycles	Time in µs	Time in ns
1	7	0.4375	437.5
2	10	0.6250	625
3	13	0.8125	812.5
4	16	1.0000	1000
255	769	48.0625	48062.5

Table 2: *Delay per loop iteration for various initial values*

9 A longer delay with four registers

```
1       ldi   r19,  4         ; for pin 2 (4 = 2^2)
2       out   0x0a, r19
3       ldi   r18,  0
4   loop:
5       eor   r18,  r19       ; 1 cycle
6       out   0x0b, r18       ; 1 cycle
7
8       ldi   r24,  0x54      ; 1 cycle
9       ldi   r25,  0x58      ; 1 cycle
10      ldi   r26,  0x14      ; 1 cycle
11      ldi   r27,  0x00      ; 1 cycle
12  delay:
13      subi  r24,  1         ; 1 cycle
14      sbci  r25,  0         ; 1 cycle
15      sbci  r26,  0         ; 1 cycle
16      sbci  r27,  0         ; 1 cycle
17      brne  delay           ; 2 cycles if jump taken, 1 cycle otherwise
18
19      rjmp  loop            ; 2 cycles
```

Listing 5 Program `led_pin2_delay4.s`: four registers (`r24` to `r27`) are used as a 32-bit counter to produce a much longer delay. Pin 2 is still toggled with each iteration of the outer loop.

To achieve a longer delay, we need more than 8 bits for the countdown counter. The program shown in Listing 5 therefore uses a 32-bit counter implemented with the four registers `r24` to `r27`. These are treated as a register group and counted down together to achieve a much longer delay than with just a single register.

A *register group* means interpreting several consecutive 8-bit registers as one common larger register, in our case a 32-bit counter. Each register stores one byte of the total value, with `r24` holding the least significant byte and `r27` the most significant byte. For example, the following assignment

```
r24 = 0x12,   r25 = 0x34,   r26 = 0x56,   r27 = 0x78
```

corresponds to the combined value

$$n = \texttt{r27:r26:r25:r24} = \texttt{0x78563412}.$$

In Listing 5, the counter is initialized in lines 8 to 11 with four `ldi` instructions. There we get

$$n = \texttt{r27:r26:r25:r24} = \texttt{0x00000200} = 512,$$

which means that the loop is executed 512 times before the state of pin 2 is changed again.

Our microprocessor has no instruction for directly decrementing a register group of four registers[4]. However, such an operation can be emulated by a combination of `subi` (subtract immediate) and `sbci` (subtract immediate with carry). This is done in lines 13 to 16 of Listing 5:

```
subi  r24,  1
sbci  r25,  0
sbci  r26,  0
sbci  r27,  0
```

Put simply, these four instructions have the effect that the entire 32-bit value in the register group `r27:r26:r25:r24` is decremented by one. As soon as the value has reached zero (i.e., all four registers contain the value `0x00`), the zero flag in the status register is set – otherwise it remains cleared.

In more detail: The `subi` instruction subtracts the value 1 from `r24` and in doing so sets the flags in the status register, among them the carry flag (C) if a borrow occurred. Two typical cases:

- **Case 1:** `r24 = 0x01` before the subtraction.
  After `subi r24, 1`, `r24 = 0x00`, and no borrow occurs; the carry flag remains cleared.
- **Case 2:** `r24 = 0x00` before the subtraction.
  After `subi r24, 1`, `r24 = 0xFF`, since the subtraction underflows. In this case, the carry flag is set.

In general: with `subi`, the carry flag is set exactly when `r24` had the value `0x00` before the subtraction, i.e., when a borrow occurs when subtracting 1.

The subsequent `sbci` instructions then each subtract the value 0 *taking into account* the carry flag. If, e.g., there was a borrow when decrementing `r24` (because it was 0 before), this borrow is propagated to `r25`, so that the group is correctly decremented by 1. The chaining of these four instructions thus emulates a correct 32-bit decrement. The zero flag is set only if the result of the entire operation is zero, i.e., all four registers contain `0x00` and there was no further borrow after the last `sbci` instruction.

9.1 Clock cycles per loop iteration

Figure 8 shows a flowchart for Listing 5. Although we have considerably more instructions than with the one-register delay, the structure is very similar: an inner loop for the delay and an outer one for toggling pin 2.

So we can determine in the same way how many clock cycles elapse until the state of pin 2 changes again. If n is the value of the register group `r27:r26:r25:r24`, then the inner loop takes

$$(n-1)\cdot 6+5$$

cycles. The outer loop contains further instructions that together take 8 cycles. Altogether, this gives:

$$T(n)=(n-1)\cdot 6+5+8=6n+7$$

[4]For a register group of two registers, e.g., `r25:r24`, there is the `SBIW` instruction (Subtract Immediate from Word).

This can be checked for a few example values:

- $n = 1 \Rightarrow T(1) = 13$ cycles $\Rightarrow 13 \cdot 62.5\,\text{ns} = 812.5\,\text{ns}$
- $n = 2 \Rightarrow T(2) = 19$ cycles $\Rightarrow 19 \cdot 62.5\,\text{ns} = 1187.5\,\text{ns}$

An even more interesting question is which value of n you have to choose so that pin 2 blinks at a frequency of 1 Hz, i.e., is HIGH for 500 ms and LOW for 500 ms. For this, the following must hold:

$$(6n + 7) \cdot 62.5\,\text{ns} = 500\,\text{ms} = 500\,000\,000\,\text{ns}$$

Solving this equation for n gives:

$$n = \frac{500\,000\,000/62.5 - 7}{6} = \frac{8\,000\,000 - 7}{6} = 1\,333\,332.166\ldots$$

Since we can only use integer values for n, we choose:

$$n = 1\,333\,332 = \texttt{0x145854}$$

This value is within our measurement accuracy and ensures that a connected LED blinks at about 1 Hz.

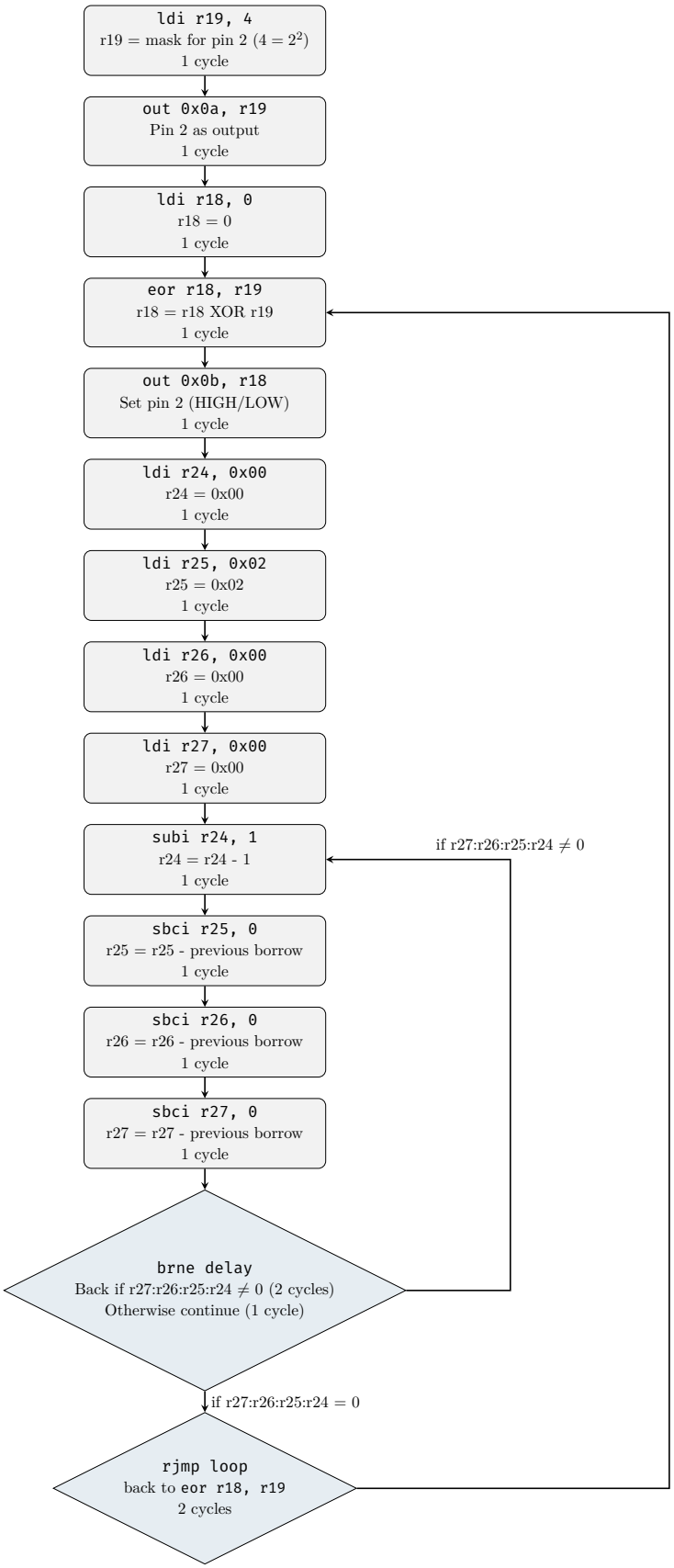


Figure 8: *Flowchart for the program* `led_pin2_delay4.s`*. The countdown uses a 32-bit register group (*`r27:r26:r25:r24`*). The diagram shows how the inner loop with* `subi/sbci/brne` *produces a longer delay before the state of pin 2 is changed again in the outer loop.*

10 Bare-metal C

```
1  static void setPinModes(unsigned char mode)
2  {
3     *(volatile unsigned char *)(0x2A) = mode;
4  }
5
6  static void digitalWriteAll(unsigned char val)
7  {
8     *(volatile unsigned char *)(0x2B) = val;
9  }
10
11 static void delay_cycles(unsigned long count)
12 {
13    do {
14       // compiler should use registers for count
15       asm volatile ("" : "+r"(count));
16       count = count - 1;
17    } while (count != 0);
18 }
19
20 int main()
21 {
22    // setup
23    unsigned char pin = 2;
24    unsigned char pinModes = 1 << pin;
25    setPinModes(pinModes);
26
27    unsigned char pinValues = 0;
28    while (1) {
29       // loop
30       pinValues ^= 1 << pin;
31       digitalWriteAll(pinValues);
32       delay_cycles((500000000 / 62.5 - 7) / 6);
33    }
34 }
```

Listing 6 C program `led_pin2_delay.c`, which produces almost the same assembly code as the handwritten version in Listing 5. The LED at pin 2 is toggled with exactly the same delay. Differences in the generated assembly code (Listing 8) are minimal and concern only irrelevant details.

What actually happens when you click "Upload" in the Arduino IDE? With the knowledge from this project, we can now answer this question – and carry out the individual steps ourselves. In this last section, we move up from machine code to C, but stay very close to the hardware: no libraries, no magic – bare metal.

10.1 A C program for a blinking LED

Listing 6 shows a C program that makes an LED connected to pin 2 blink at a frequency of 1 Hz. If you look at the code, you may see that there is some magic involved after all. The code in lines 1 to 18 is not for the faint of heart and requires not only a somewhat deeper understanding of the C programming language and of how a C compiler generates machine code from it, but also knowledge of the microprocessor. Here, for example, the addresses `0x2A` and `0x2B` are used to configure pin 2 as an output and to set it HIGH or LOW, respectively.

But here is the point: an end user does not need to understand how the code in lines 1 to 18 works in order to use it. Because ultimately, all it does is provide easy-to-use functions:

- `setPinModes` – to configure pins as inputs or outputs,
- `digitalWriteAll` – to set pins HIGH or LOW,
- `delay_cycles` – to wait for a certain number of clock cycles.

The implementation of these functions can be hidden in a library; you then only need to know what these functions do and how to use them. For Arduino, there is very good documentation for this.

As a user, you then only have to write the code in lines 20 to 34 yourself. And even this part can be simplified further.

So the point here is **not** to understand this particular C code in detail – after all, we will use the Arduino development environment later, and with it things are much easier. The aim here is merely to show that, in principle, a few lines of C code can implement exactly what we programmed in assembly language before: a blinking LED, directly on the hardware, without libraries, very close to the metal.

10.2 Toolchain

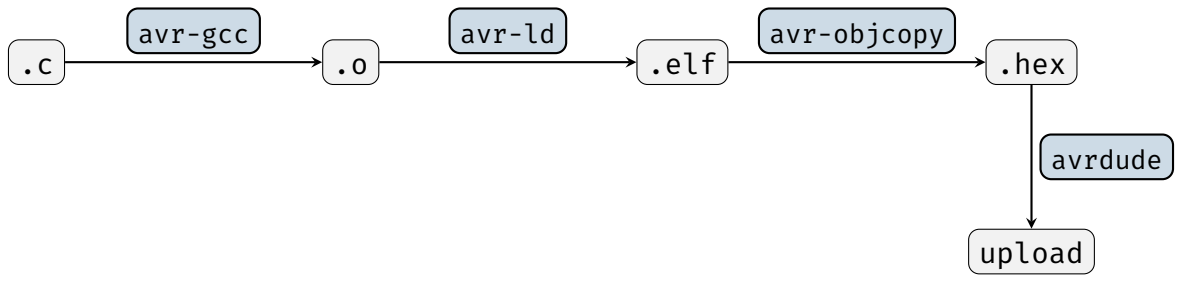


Figure 9: *Dependency graph for generating a* `.hex` *file and the subsequent upload. The tools* `avr-gcc`*,* `avr-ld`*, and* `avr-objcopy` *generate the* `.hex` *file, which is finally transferred to the microcontroller by* `avrdude`*.*

The toolchain has now changed slightly: the C compiler can generate machine code directly from the C code in the form of an object file. This `.o` file is then processed, as before, by the linker and other tools until a `.hex` file is produced at the end. An overview of this process is shown in Figure 9. Of course, you can use a Makefile as in Listing 7 so

that you do not have to type every command every time. And with that, the secret is actually already revealed:

What happens when you click "Upload"?

If you type in a program like the one in Listing 6 and press the Upload button, a command like
`make upload HEX=led_pin2_delay.hex`
is executed internally, which automatically runs through the complete toolchain.

We can also enter this command manually on the command line and observe what happens:

```
MCL:c lehn$ make upload HEX=led_pin2_delay.hex
avr-gcc -mmcu=atmega328p -c -Os -o led_pin2_delay.o led_pin2_delay.c
avr-ld -mavr5 -o led_pin2_delay.elf led_pin2_delay.o
avr-objcopy -O ihex -R .eeprom led_pin2_delay.elf led_pin2_delay.hex
avrdude -c arduino -P /dev/cu.usbserial-01C5FB3D -p atmega328p -U flash:w:
   led_pin2_delay.hex:i
Reading 30 bytes for flash from input file led_pin2_delay.hex
Writing 30 bytes to flash
Writing | ################################################## | 100% 0.03 s
Reading | ################################################## | 100% 0.01 s
30 bytes of flash verified

Avrdude done. Thank you.
rm led_pin2_delay.elf led_pin2_delay.o
```

10.3 Generating readable assembly instead of machine code

In the past, C compilers generated only assembly code, which was then translated into machine code by an assembler. Today's compilers usually skip this step and generate machine code directly, since an assembler is practically already integrated into the compiler.

If you like, however, you can still have the generated assembly code output – and that is exactly what is interesting in our case. Our Makefile helps us call the compiler with the options required for this. With the command

```
MCL:c lehn$ make led_pin2_delay.s
avr-gcc -mmcu=atmega328p -S -Os -o led_pin2_delay.s led_pin2_delay.c
```

the C code `led_pin2_delay.c` is translated into assembly code (`led_pin2_delay.s`).

Listing 8 shows the assembly code generated by the C compiler `avr-gcc`. Only lines 16 to 32 are relevant here. In fact, you could delete all the other lines and generate a `.hex` file from the remaining lines that would not differ in functionality.

```
# This version processes assembly programs (*.s)
# and C programs (*.c)

ASM_SOURCES := $(wildcard *.s)
C_SOURCES := $(wildcard *.c)
OBJECTS := $(ASM_SOURCES:.s=.o) $(C_SOURCES:.c=.o)
ELFS  := $(OBJECTS:.o=.elf)
HEXS  := $(ELFS:.elf=.hex)

all: $(HEXS)

%.o: %.c
      avr-gcc -mmcu=atmega328p -c -Os -o $@ $<

%.s: %.c
      avr-gcc -mmcu=atmega328p -S -Os -o $@ $<

%.o: %.s
      avr-as -o $@ $<

%.elf: %.o
      avr-ld -mavr5 -o $@ $<

%.hex: %.elf
      avr-objcopy -O ihex -R .eeprom $< $@

clean:
      $(RM) $(OBJECTS) $(ELFS) $(HEXS)

ifeq ($(MAKECMDGOALS),upload)
ifeq ($(shell uname), Linux)
PORT := $(shell ls /dev/ttyUSB* 2>/dev/null | head -n 1)
else
PORT := $(shell ls /dev/cu.usb* 2>/dev/null | head -n 1)
endif # uname
endif # MAKECMDGOALS == upload

upload: $(HEX)
      @if [ -z "$(HEX)" ]; then \
         echo "Usage: make upload HEX=<filename.hex>"; \
         false; \
      fi
      @if [ -z "$(PORT)" ]; then \
         echo "no USB device found"; \
         false; \
      fi
      avrdude -c arduino -P $(PORT) -p atmega328p -U flash:w:$(HEX):i
```

Listing 7 `Makefile` for translating C and assembly programs on Linux and macOS. This version processes both `.c` files and `.s` files and automatically generates matching `.hex` files. An optional target `upload` detects the serial interface automatically and transfers the program via `avrdude`.

10.4 Comparison with our assembly program

```
1       .file "led_pin2.c"
2  __SP_H__ = 0x3e
3  __SP_L__ = 0x3d
4  __SREG__ = 0x3f
5  __tmp_reg__ = 0
6  __zero_reg__ = 1
7       .text
8       .section    .text.startup,"ax",@progbits
9  .global main
10      .type main, @function
11 main:
12 /* prologue: function */
13 /* frame size = 0 */
14 /* stack size = 0 */
15 .L__stack_usage = 0
16      ldi r18,lo8(4)
17      out 0xa,r18
18      ldi r18,0
19      ldi r19,lo8(4)
20 .L3:
21      eor r18,r19
22      out 0xb,r18
23      ldi r24,lo8(84)
24      ldi r25,lo8(88)
25      ldi r26,lo8(20)
26      ldi r27,0
27 .L2:
28      sbiw r24, 1
29      sbci r26, 0
30      sbci r27, 0
31      brne .L2
32      rjmp .L3
33      .size main, .-main
34      .ident "GCC: (Homebrew AVR GCC 14.2.0) 14.2.0"
```

Listing 8 Assembly code `led_pin2_delay.s`, generated automatically by `avr-gcc` from the C program in Listing 6. The code differs from the handwritten version in Listing 5 in only two irrelevant details; in particular, the timing remains completely identical.

Let us go through the differences from our handwritten assembly program step by step:

1. Lines 16 to 19 read:

   ```
   ldi r18,lo8(4)
   out 0xa,r18
   ldi r18,0
   ldi r19,lo8(4)
   ```

This is equivalent to:

```
ldi r18,4
out 0xa,r18
ldi r18,0
ldi r19,4
```

So the compiler has generated an unnecessary instruction here. Cleverer (and shorter) would be:

```
ldi r19,4
out 0xa,r19
ldi r18,0
```

And that is exactly what we did in our program.

2. Lines 20 to 22 read:

```
.L3:
eor r18,r19
out 0xb,r18
```

Our label is called `loop`, the compiler's is called `.L3`. The rest is identical. What you call a label does not matter – humans prefer meaningful names, compilers often use `.L` followed by a consecutive number.

3. Lines 23 to 26 read:

```
ldi r24,lo8(84)
ldi r25,lo8(88)
ldi r26,lo8(20)
ldi r27,0
```

This is equivalent to:

```
ldi r24, 84
ldi r25, 88
ldi r26, 20
ldi r27, 0
```

Writing the values in hexadecimal, as we do, gives:

```
ldi r24, 0x54
ldi r25, 0x58
ldi r26, 0x14
ldi r27, 0x00
```

4. Lines 27 to 32 read:

```
.L2:
sbiw r24, 1
sbci r26, 0
sbci r27, 0
brne .L2
rjmp .L3
```

Our label is called `delay` instead of `.L2`, since we are humans. Another difference is the first instruction:

```
sbiw r24, 1
```

It takes 2 cycles and decrements the register pair `r25:r24`. So it has the same effect as:

```
subi r24, 1
sbci r25, 0
```

The compiler uses `sbiw` here because this instruction – although it takes exactly as many cycles as our variant with `subi` and `sbci` – reduces the code size. `sbiw` occupies only 2 bytes in flash, whereas two separate instructions would require 4 bytes in total. Especially on microcontrollers with limited program memory, this is an important advantage.

11 More fun with the C code

We have seen that the C code is functionally equivalent to our assembly code. Now, of course, you can also experiment with the C code. For example, Listing 6 contains the line:

```
delay_cycles((500000000 / 62.5 - 7) / 6);
```

This has the effect that pin 2 is alternately HIGH for 500 ms and then LOW for 500 ms. The number `n = 500000000` specifies the desired delay in nanoseconds.

Of course, you can choose other values here as well – including very small ones such as `n = 1` or `n = 2` – and use the oscilloscope to verify that the behavior changes in exactly the same way as in our assembly program.